

Bom dia, professor. Vou apresentar minha dissertação de mestrado — *Explicando Decisões de LLMs via Otimização Inversa* — orientada pelo Prof. Raul Fonseca Neto. A apresentação está organizada em três blocos auto-contidos: no Bloco 1 o LLM é a fonte de decisões; no Bloco 2 o LLM é o aprendiz (Problema E); no Bloco 3 aplicamos tudo na base real peso $\times$ altura. Os números desta apresentação vêm de execução completa de produção com 3 sementes e 3 repetições.

LLMs são caixas-pretas. Pedir explicação em linguagem natural dá racionalização, não causa real. Em aplicações críticas isso é insuficiente. A pergunta central: o LLM tem um processo decisório estável e aprendível? Em vez de pedir explicação, observamos as decisões e recuperamos o critério por trás delas via otimização inversa.

Cinco hipóteses. H1 e H2 no Bloco 1: critério estável e few-shot amplifica. H3 e H4 no Bloco 2: LLM aprende via in-context learning e exemplos hard deveriam ensinar mais que easy. H5 transversal. Os achados desta execução mostram que H4 é refutada em `n_shot` pequeno mas revertida em `n_shot=40` — diagnóstico mais nuançado que o esperado.

Trato cada classificação do LLM como saída de um classificador nearest-centroid ponderado por Mahalanobis diagonal. Dados pares ponto-rótulo do LLM, encontro $W \geq 0$ que reproduz suas decisões. Uso dois algoritmos independentes — Perceptron Estruturado e NNLS. Se ambos chegam ao mesmo W , a evidência não depende do método.

A distância é $d_W(x, c) = \sum w_j (x_j - c_j)^2$. Classificador: argmin sobre as classes. Mahalanobis diagonal corresponde a Σ^{-1} quando features são independentes. $O(d)$ parâmetros e interpretabilidade direta. Destaco a nomenclatura das fases — Problema D é meia-lua não-linear (Bloco 1), Problema E é LLM aprendiz (Bloco 2).

Esta é a execução de produção completa: 3 sementes (42, 123, 7), 3 repetições por configuração, n_shots de 0 a 40, 4 estratégias, 3 experts. 9 observações por célula para bootstrap CI. Modelo gpt-4o-mini com temperatura zero. Hiperparâmetros do Perceptron: $\eta = 0,001$ e $C = 1,0$, conforme CILAMCE 2017.

Mostro o pseudocódigo completo do Perceptron Estruturado, do artigo de Coelho, Borges e Fonseca Neto, CILAMCE 2017, Seção 5.2. A estrutura tem duas camadas: busca binária no γ ótimo (loop externo, Equação 31) e Perceptron interno que corrige w por ponto violado (Equação 29). Os hiperparâmetros do artigo são $\eta = 0,001$ e C entre 0,1 e 1,0; usamos 1,0. Peço atenção para a projeção em $w \geq 0$ que garante semidefinição positiva da matriz diagonal. O atributo `gamma_history` capturado em cada iteração fornece o diagnóstico da busca binária que o professor pediu na reunião de 30/04 em torno de $\sim 520s$ — a figura aparece no slide 83.

Mostro o pseudocódigo do NNLS de Lawson e Hanson de 1974. Construção da matriz A com diferenças de quadrados de distâncias aos centróides e vetor b de uns (margem-alvo). Chamada única para `scipy.optimize.nnls(A, b)` resolve a otimização $\min \|Aw - b\|^2$ com $w \geq 0$. Vantagens sobre Perceptron: zero hiperparâmetros, determinístico (reproduzível bit-a-bit), otimalidade global garantida do problema convexo QP. Cosseno típico com o W do Perceptron acima de 0,97 — validação cruzada do método.

Pipeline com 5 camadas de proteção: parser de 7 camadas, até 5 tentativas para respostas malformadas, fallback determinístico por hash MD5 sem viés geométrico, concorrência assíncrona com semáforo de 10 que reduz tempo em uma ordem de magnitude, timeout de 60s com wrap de 90s contra travamento.

Entramos no Bloco 1, onde o LLM é a fonte de decisões. O LLM rotula, nós aprendemos a métrica W . Cobrirei os problemas A, B, C lineares e o D meia-lua. Hipóteses: H_1 e H_2 .

Apresento textualmente os 4 problemas que serão detalhados nos próximos slides. Problema A é a bancada de treinamento, gaussianas lineares com centróides em $(-2, 0)$ e $(2, 0)$, $n = 150$. Problema B é rotação horária agressiva, centróides em $(-2; +1,5)$ e $(2; -1,5)$, $n = 100$. Problema C é a rotação anti-horária pareada simetricamente, centróides em $(-2; -1,5)$ e $(2; +1,5)$. Problema D é a meia-lua sintética do `make_moons` com $n = 150$ e ruído 0,15 — problema-controle para R3/R4. Os primeiros 3 são lineares com transferência desafiadora; o 4º é o teste explícito de não-linearidade.

A bancada de treinamento. `make_blobs`, centróides em $(-2, 0)$ e $(2, 0)$, desvio padrão 1,2 isotrópico, 150 amostras. Linearmente separável em x_1 . Fácil para o LLM.

Testa transferência em geometria deslocada agressivamente. Centróides em $(-2; +1,5)$ e $(2; -1,5)$ — rotação horária. Magnitude $\pm 1,5$, simetricamente pareada com C. Mais agressiva que a versão anterior de $\pm 0,8$.

Mesma magnitude de B mas direção oposta: anti-horária. Centróides em $(-2; -1,5)$ e $(2; +1,5)$. O orientador pediu esta orientação oposta na reunião de 30 de abril, porque o C anterior parecia cópia de B. Agora B e C são pares: mesma severidade, direções opostas.

Meia-lua entra no Bloco 1 como problema-controle não-linear. `make_moons` com $n = 150$, ruído 0,15. O orientador disse na reunião: "a base da meia-lua você acha em qualquer lugar aí". É onde esperamos que augmentação R3/R4 ajude.

Primeira parte do sanity check. Pergunta: se W verdadeiro fosse conhecido, Perceptron e NNLS o recuperariam? Gero dados sintéticos com W oráculo em três configurações anisotrópicas e aprendo $\hat{W}$ via os dois algoritmos. A tabela mostra resultado médio em 3 seeds. Em $x2_dom$ (razão verdadeira 16), NNLS chega a razão infinita (acerta a direção perfeitamente), cosseno 0,999; Perceptron normaliza para razão 1,32, cosseno 0,824. Mesmo padrão em $x1_dom$ e forte $aniso$: NNLS preserva razões extremas, Perceptron tende a centralizar. Mensagem-chave: ambos atingem fidelidade 100% — o sinal classificatório foi capturado — mas NNLS é fiel à magnitude exata do W oráculo.

Aqui o gráfico grande do painel 2x2: cosseno por configuração, fidelidade por configuração, razão w_1/w_2 recuperada vs verdadeira, e scatter de W recuperado vs oráculo. Visualmente fica claro o padrão da tabela anterior: NNLS pontos próximos da diagonal (acerta razão), Perceptron pontos concentrados perto do meio (normaliza). Mensagem para a banca: a discrepância entre algoritmos não é bug — é diferença metodológica conhecida. NNLS é otim global, Perceptron é gradient descent com regularização implícita pela margem.

Segunda parte do sanity check, agora sobre transferência. Pergunta dual: se aprendo W em uma geometria, ele transfere para outra? Aprendo W em $x2_dom$, aplico em $x1_dom$, forte_aniso, Problemas A e E. A tabela mostra cosseno alto entre o W aprendido em origem e o W oráculo no destino. Em problemas com geometria similar, transferencia funciona; em Problema A (gaussianas isotrópicas), cosseno cai para 0,75 — a geometria é suficientemente diferente. Mensagem: implicação direta para Problemas B/C — sempre recalculamos centróides LOCAIS, não reaproveitamos do Problema A. A métrica não é independente da geometria do problema.

Quarto slide do Oracle. Gráfico grande mostrando os sub-painéis de transferência: cada expert origem com suas curvas para os experts destino. Conclusão do sanity check Oracle: ambos algoritmos passam — recuperam W ($\sim$ fidelidade 100%) e transferem com cosseno alto entre geometrias similares. Caminho está livre para aplicar o método ao LLM nas próximas seções.

A Problema A: LLM recebe 150 pontos em zero-shot, responde A ou B. As respostas formam (X, y_{LLM}) . Centróides recalculados a partir das respostas do LLM — ancoramos no que o LLM ACHA serem as classes. Perceptron e NNLS aprendem W . Medimos fidelidade.

Com as 3 seeds, a fidelidade média da Problema A foi 75,1% com desvio 8,4 pontos percentuais. Por seed: 72,3, 75,2, 78,2%. A acurácia do LLM contra ground truth foi apenas 40% — o LLM inverteu classes em vários casos. Mesmo assim, a métrica reproduz 75% das decisões erradas, demonstrando que H1 mede consistência interna, não correção.

O painel mostra a distribuição do W aprendido pelos dois algoritmos nas três seeds. Boxplot, ratio w_1/w_2 , scatter de W . Estabilidade qualitativa entre seeds.

Perceptron e NNLS lado a lado: fidelidade, fases B/C, scatter de W . Mensagem: os dois algoritmos são congruentes com cosseno acima de 0,95. Robustez ao método de estimação.

Aqui faço uma decisão metodológica explícita para a banca. O trabalho mede consistência interna do critério decisório do LLM: fidelidade na Problema A (quanto $\hat{W}$ reproduz y_{LLM}), consistência em B/C (quanto $\hat{W}_A$ prevê y_{LLM} em novos problemas) e acurácia LLM-perito na Problema E. Decisão: não comparamos com ground truth nos problemas sintéticos. Razão: H1 testa se o LLM tem critério estável, não se ele está objetivamente correto. Um LLM pode inverter classes e ainda assim ser internamente consistente. Exceção: Bloco 3 (peso $\times$ altura) reportará acurácia vs rótulo real, atendendo solicitação explícita do orientador no e-mail das 22:04, ponto 4 — lá há ground truth significativo homem/mulher e classificador ótimo conhecido.

Aqui apresento os pesos $W = (w_1, w_2)$ aprendidos para o Problema A, separados por seed e por algoritmo (Perceptron e NNLS). Média das 3 repetições por seed. Os achados-chave:

Perceptron Estruturado: seed 42 $W = (0,0008, 0,0031)$ razão 0,245; seed 123 $W = (0,0024, 0,0094)$ razão 0,251; seed 7 $W = (0,0256, 0,0725)$ razão 0,353.

NNLS: seed 42 $W = (0,0355, 0,1414)$ razão 0,251; seed 123 $W = (0,0467, 0,1493)$ razão 0,313; seed 7 $W = (0,0381, 0,1492)$ razão 0,255.

Antes das observações, vale esclarecer o que é *fidelidade na Problema A*: é a porcentagem de pontos do Problema A em que o rótulo atribuído pela métrica $\hat{W}$ coincide com o rótulo que o LLM atribuiu. É medida *in-sample*, ou seja, nos mesmos pontos usados para estimar $\hat{W}$. Funciona como sanidade: se fosse baixa, o algoritmo não teria conseguido ajustar uma métrica diagonal aos rótulos do LLM, e tudo que viria depois (consistência em B/C) perderia o sentido. Observações importantes para a banca: as magnitudes diferem entre algoritmos (Perceptron uma ordem de grandeza menor que NNLS), mas as razões w_1/w_2 são similares — isso confirma que a otimização inversa é invariante à escala. As razões em torno de 0,25–0,35 revelam que o LLM pondera x_2 cerca de três a quatro vezes mais que x_1 , apesar do Problema A ser linearmente separável em x_1 — pequena anisotropia decisória do LLM. Por fim, a fidelidade na Problema A é quase idêntica entre os dois algoritmos, reforçando que a estimação de W é robusta.

Apresento a tabela das Fases B e C por seed e por algoritmo, média de 3 repetições por seed, zero-shot. Antes dos números, esclareço o que é consistência: é a porcentagem de pontos em que o rótulo atribuído pela métrica $\hat{W}_A$ — aquela que aprendemos no Problema A — coincide com o rótulo do LLM nos Problemas B e C, que têm geometrias rotacionadas. O Kappa de Cohen corrige por concordância ao acaso. Perceptron por seed: 42 dá B 67,3% / κ 0,26 e C 91,0% / 0,81; 123 dá B 65,3% / 0,25 e C 90,3% / 0,80; 7 dá B 76,0% / 0,47 e C 86,3% / 0,71. Média Perceptron: B 69,5% / 0,33 e C 89,2% / 0,77. NNLS por seed: 42 dá B 68,0% / 0,29 e C 90,0% / 0,79; 123 dá B 67,0% / 0,29 e C 89,0% / 0,77; 7 dá B 76,0% / 0,47 e C 84,0% / 0,66. Média NNLS: B 70,3% / 0,35 e C 87,7% / 0,74. Observações para a banca: H1 sustentada; em todas as seeds o Kappa em C é bem maior que em B, indicando que a rotação anti-horária preserva melhor o critério decisório do LLM. A seed 7 se comporta como outlier sutil — em B sobe (76%) mas em C cai mais que as outras. Nota metodológica: usamos centróides LOCAIS (recalculados em cada problema a partir de y_{LLM}), mais fiéis à decisão real; TRANSFERIDOS testados como ablativo (locais 65%, transferidos 77% em B).

Agora um painel visual complementar à tabela anterior, com seed 42 como representante — a tabela já mostrou que a seed 42 fica próxima da média. À esquerda o Problema B, à direita o C. Cada ponto colorido em verde é um acerto da métrica $\hat{W}_A$ contra o rótulo do LLM, e em vermelho é um erro. O importante é o padrão geométrico dos erros: em B eles se concentram na faixa central, onde as duas gaussianas rotacionadas se sobrepõem — a métrica $\hat{W}_A$ aprendida no Problema A foi calibrada para uma geometria sem rotação, e a rotação horária de $\pm 1,5$ deslocou a fronteira para uma região onde ela não distingue mais bem. Já em C os erros aparecem esparsos, isolados, sem concentração em nenhuma região. Visualmente fica claro por que o Kappa de C é cerca de três vezes maior que o de B mesmo com magnitude de rotação idêntica: a rotação anti-horária preserva muito melhor o critério decisório do LLM. Esse mapa converte a estatística $\kappa_C \gg \kappa_B$ em algo geométrico para a banca.

Tabela por seed comparando R2, R3 e R4 apenas em dois problemas: o A (linear, com três seeds) e a meia-lua (não-linear, com um dataset de seed 42). B e C foram omitidos porque são lineares como A — augmentação R3/R4 seria redundante e não foi rodada. Peso $\times$ altura não aparece aqui, está reservado para o Bloco 3. Antes da leitura, recapitulo: R2 é o espaço original (x_1, x_2); R3 acrescenta o produto $x_1 \cdot x_2$ para capturar interação; R4 acrescenta x_1^2 e x_2^2 para capturar curvatura. A métrica é estimada nesses espaços e medida *in-sample* (fidelidade contra os rótulos do LLM). No Problema A, Perceptron por seed: 42 dá 82,0/84,7/84,7; 123 dá 86,7/75,3/84,7; 7 dá 86,0/84,0/90,0. Média Perceptron 84,9/81,3/86,4. NNLS por seed: 42 82,0/84,0/86,0; 123 86,7/87,3/88,7; 7 87,3/86,0/92,0. Média NNLS 85,3/85,8/88,9. Em meia-lua (só seed 42): Perceptron 68,6/74,3/75,2 e NNLS 68,6/82,9/76,2. Leitura para a banca: em A linear os ganhos R3/R4 ficam dentro do ruído entre seeds — a seed 123 do Perceptron até cai em R3, evidenciando que augmentação não ajuda quando o problema já é linearmente separável. Na meia-lua, ao contrário, o NNLS pula de 68,6% em R2 para 82,9% em R3, o ganho mais expressivo da tabela — o termo cruzado $x_1 x_2$ recupera a curvatura que a métrica diagonal em $\mathbb{R}^2$ era incapaz de representar. Limitação metodológica importante de citar: o pipeline na verdade rodou os três seeds (42, 123 e 7), mas em zero-shot o LLM colapsou todos os 105 pontos em uma única classe nos seeds 123 e 7 — o log da execução registra “classe única no LLM, pulando”. Sem dois grupos distintos, não há sinal de separação para a estimação inversa ajustar, e o pipeline corretamente abortou esses seeds. Apenas a seed 42 sobreviveu. Isso é um achado em si: a meia-lua é o problema mais sensível ao posicionamento das luas geradas pelo `make_moons`, e em duas das três configurações o gpt-4o-mini, sem few-shot, não percebeu que existiam duas classes distintas. Documenta uma fragilidade do zero-shot nesse tipo de geometria.

Aqui apresento a tabela do experimento de inversão de ordem das classes. Para cada par semântico — letras, números, semantical positivo/negativo e cores — rodei o mesmo prompt em duas direções: ordem direta e ordem invertida ($n_shot=10$, média de 3 seeds vezes 3 repetições). Peço atenção para a coluna “Fid. A”: em A/B caiu de 84,9% para 69,1% quando invertemos para B/A; em 0/1 subiu de 62,2% para 72,4% ao virar 1/0; em positivo/negativo a inversão manteve fidelidade alta (98,4% contra 95,1%); em cores a variação foi pequena (84,2% contra 82,0%). Leitura para a banca: existe sim um viés de ordem, e ele varia entre ~ 2 e ~ 15 pontos percentuais dependendo do tipo semântico. Letras e números são mais sensíveis que cores e rótulos semânticos — o LLM parece ter um vínculo mais forte entre certas letras/números e “classe positiva” que persiste mesmo quando trocamos a ordem do prompt. Porém, o viés não é dominante: o critério decisório geral se mantém (consistências B e C continuam altas). Portanto o efeito existe, está documentado, mas não compromete os achados principais.

Complemento visual da tabela anterior. O gráfico `bloco1_12_class_order_bias.png` mostra, para cada par semântico, as barras lado a lado da consistência/Fidelidade em ordem direta versus invertida. Visualmente, a banca pode notar que os pares “A/B vs B/A” e “0/1 vs 1/0” são os de maior amplitude — corroborando a leitura numérica de que letras e números disparam viés maior que cores e rótulos semânticos. Para Pos/Neg e Azul/Vermelho as barras são quase idênticas. Conclusão: o viés de ordem está presente, principalmente em pares letra/número, mas não explica os achados principais do trabalho.

Antes de mostrar os números, exibo o conteúdo de cada variante para o ponto exemplo $x_1 = 1,23$, $x_2 = -0,45$ com classes A/B. O *default* é o template baseline, sem floreios, só pede a classe. O *geometric* acrescenta a frase explicando que os pontos vivem em um espaço métrico euclidiano e que a classificação deve se basear em proximidade espacial aos centros dos clusters. O *CoT* pede que o modelo pense passo a passo e dê a resposta final na última linha. O *tabular* formata as coordenadas em tabela markdown ao invés de prosa. Essas são perturbações pequenas mas controladas; o próximo slide mostra que essas perturbações mudam de fato a métrica recuperada — então o prompt não é neutro.

Aqui apresento a tabela com os resultados das 4 variantes de prompt rodadas nas Problemas A–C completas em todas as seeds. Em zero-shot, média de 3 seeds vezes 3 repetições, configuração A/B com features x_1/x_2 . A variante *default* é o template baseline. *Geometric* acrescenta contexto espacial explícito (eixos e regiões). *CoT* é chain-of-thought, pede raciocínio passo a passo. *Tabular* apresenta as coordenadas em tabela markdown. Os números: default fid A 84,9%, cons B 69,6% / κ_B 0,33, cons C 89,2% / κ_C 0,77. Geometric cai para 79,1 / 68,2 / 0,14 / 75,3 / 0,35. CoT cai mais ainda, 71,8 / 60,4 / 0,14 / 79,8 / 0,60. Tabular é o pior: 64,4 / 64,3 / 0,15 / 72,7 / 0,20. Leitura para a banca: ao contrário do que se esperava, prompts mais elaborados *não melhoram* a coerência decisória do LLM — pioram. O default puro é que dá o melhor Fid. A. CoT, especificamente, não ajuda em problemas geométricos simples como os nossos — talvez porque introduza ruído verbal sem informação adicional. A tabela markdown degrada ainda mais, provavelmente porque desloca o LLM para um modo de leitura tabular onde os pares (x_1 , x_2) perdem a intuição espacial. Sensibilidade ao prompt é portanto *real e não desprezível* (queda de até 20 p.p. em Fid. A) — adotamos o default como referência e os outros como controle ablativo.

Complemento visual da tabela anterior. A imagem `bloco1_13_prompt_variants.png` mostra, para cada variante de prompt, o scatter dos pesos W aprendidos e as métricas em A, B e C lado a lado. Visualmente fica claro que os pontos de *default* se separam dos pontos de *geometric*, *CoT* e *tabular*. Mensagem para a banca: a configuração do prompt não é neutra — escolhas que parecem inocentes (passar coordenadas em prosa versus tabela versus pedir CoT) modificam mensuravelmente o W que conseguimos recuperar. Por isso a apresentação toda foi padronizada no *default*, e este experimento serve como controle ablativo documentado.

Apresento a tabela com o efeito dos nomes sobre o W recuperado, dividida em duas partes. (a) Classes em $n_shot=10$ com features x_1/x_2 : A/B dá Fid. A 84,9% e cons B/C 82,5/96,0%; 0/1 colapsa Fid. A para 62,2%; Positivo/Negativo infla Fid. A para 98,4% mas degrada κ_B para 0,48; Azul/Vermelho fica próximo de A/B. Semânticas não são neutras — o LLM parece ancorar no significado da palavra. (b) Features em zero-shot, classes A/B: x_1/x_2 (baseline) Fid. A 84,9%, κ_B 0,33, κ_C 0,77. *altura/peso* desmorona: 70,7%, $\kappa_B = -0,09$ — pior que aleatório. *feature_1/feature_2* fica em meio termo (80%, 0,39, 0,61). Mensagem para a banca: prior cultural sobreescreve o critério decisório em features semânticas; o LLM impõe correlação altura/peso sobre dados sintéticos e quebra a consistência. Por isso adotamos x_1/x_2 como baseline em todo o Bloco 1.

Complemento visual: dois painéis lado a lado. À esquerda `bloco1_14a_class_names_effect.png` com o efeito de mudar nome de classe. À direita `bloco1_14b_feature_names_effect.png` com o efeito de mudar nome de feature. Visualmente fica claro que o impacto em features (b) é bem mais pronunciado que em classes (a) — argumento concreto para usar nomes neutros (x_1/x_2 , A/B) em todo o Bloco 1.

Para `n_shot` maior que zero, o prompt inclui exemplos rotulados antes do ponto-alvo. Mostro o template literal para `n_shot=5`: o LLM vê cinco exemplos com x_1 , x_2 e rótulo, e depois o ponto novo a classificar. As variantes `geometric`, `CoT` e `tabular` já foram cobertas no slide 31, portanto aqui só exibo o caso default com cinco exemplos para a banca ver como o few-shot é construído na prática.

Fechando o Bloco 1. H1 sustentada parcialmente: Kappa C de 0,49 é substancial, mas Kappa B de 0,19 é modesto. Fidelidade da Problema A em torno de 75%. Acurácia do LLM contra real apenas 40% — LLM inverte classes mas continua consistente. R3/R4 agrega só na meia-lua. Vieses marginais.

Bloco 2: inversão completa de papéis. Um perito externo rotula, o LLM tenta reproduzir o critério a partir de exemplos few-shot. É a Problema E. Problemas: E (linear com W expert) e F (meia-lua com perito = ground truth). Hipóteses: H3, H4, H5.

Mostro o problema E linear com fronteira do expert quase horizontal, porque w_2 é cinco vezes maior que w_1 . A imagem mostra fronteira de decisão, ground truth e margem.

Problema E é o antigo "Problema D" da Problema D antiga, agora renomeado. Linear, perito anisotrópico $W = [0,3,1,5]$. Fronteira quase horizontal. LLM precisa capturar o padrão a partir de exemplos.

F usa a meia-lua com ground truth como perito. LLM vê apenas (x_1, x_2) , sem augmentação no prompt. Pergunta: consegue aprender a fronteira em S apenas com exemplos? É o teste mais duro de H3.

Passo a passo: perito rotula os 150 pontos; divido em treino e teste; para cada n_{shot} de 0 a 40, seleciono exemplos via estratégia; envio exemplos + ponto de teste em prompt few-shot; meço acurácia, Kappa, F1 contra perito.

Apresento a tabela consolidada dos resultados da Problema E em que o LLM aprende com o perito linear $W = [0,3, 1,5]$ (x2 dominante). Média de 3 seeds vezes 3 repetições. Acurácia e Kappa do LLM contra o perito, separados por estratégia (easy, hard, mixed, random) e por n_shot (0 a 40). Em zero-shot, todas as estratégias degeneram para o mesmo número (60,4% / 0,22) — sem exemplos não há o que escolher. Em $n_shot=5$, easy já salta para 85,6% enquanto hard fica em 61,9%. Em $n_shot=20$ todos sobem, mas easy ainda lidera com 90,6%. A inversão acontece em $n_shot=40$: hard salta para 97,8% e ultrapassa easy (88,8%). Mensagem para a banca: H2 (few-shot amplifica) está fortemente sustentada — saltos de quase 40 p.p. só por adicionar exemplos. E a H4 (exemplos difíceis ajudam mais) sai como *refutação nuançada*: refutada em regime de poucos exemplos, mas confirmada em $n_shot=40$. Estratégia não é monotonicamente boa, depende do regime de quantidade de exemplos — esse é um achado novo que vale destacar.

Quatro estratégias: easy alta margem, hard baixa margem, mixed 50/50, random baseline. A intuição de H4 era que hard ensina mais. Vamos ver que é mais complexo que isso.

Curvas de acurácia versus n_shot para cada estratégia, média de 3 seeds vezes 3 repetições. O LLM melhora em todas as estratégias — H3 sustentada.

Em modelos com gradiente, hard é informativo. LLM in-context não tem gradiente; usa exemplos como âncoras. Easy são pivots claros: "é isto que A parece". Hard são confusos. Com muitos exemplos, há contexto suficiente para hard ajustar fino o critério. Achado novo desta execução: dinâmica muda com `n_shot`.

Testei três experts: aniso_x2 com $W = [0,3,1,5]$ fronteira horizontal, aniso_x1 oposto, e euclidean. LLM aprende todos, mas Euclidean costuma liderar — critério mais natural.

Diluição testa se easy dilui o sinal de hard fixos. Fixei 4 hard e adicionei easy progressivamente. Curva monotônica crescente — easy ajuda sempre, não há diluição. Reforça o achado.

Aqui mostro a tabela do experimento de viés de ordem. Fixei a estratégia *mixed* e os mesmos exemplos selecionados; variei apenas a ordem em que aparecem no prompt. Testei quatro ordenações: `class0_first` (todos da classe 0 primeiro), `class1_first` (todos da classe 1 primeiro), `shuffled` (ordem aleatória) e `alternating` (alternando 0, 1, 0, 1). Roda em `n_shot = 5, 10` e `20`. Em `n_shot=5`: amplitude máxima de 3,6 p.p. entre ordenações. Em `n_shot=10`: amplitude sobe um pouco para 7,9 p.p. (`shuffled` lidera com 87,2%, `class0_first` é o pior com 79,3%). Em `n_shot=20`: amplitude colapsa para 1,3 p.p. — todos próximos de 86%. Leitura para a banca: existe sim algum efeito de ordenação em `n_shot` intermediário, mas é marginal (menor que 8 p.p. no pior caso) e some quando se aumenta o número de exemplos. Recency bias não explica os achados principais; o critério decisório do LLM é robusto à ordem.

Complemento visual da tabela anterior. A imagem `bloco2_07_example_order.png` mostra, para cada `n_shot`, as barras das quatro ordenações lado a lado. Visualmente fica claro que as barras são parecidas dentro de cada `n_shot` — corrobora a leitura numérica de que recency bias é marginal.

LLM aprende via few-shot na meia-lua não-linear. Perito = ground truth. LLM vê (x_1, x_2) sem augmentação. Acurácia vs $\hat{W}$ pula de 76% em zero-shot para 87% em 10-shot — ganho de 11 p.p. Em 20-shot cai para 78%, em 40-shot volta a 84%. Comportamento não-monotônico — plateau com variação.

O ganho zero-shot → 10-shot na meia-lua é cerca de 11 pontos percentuais de acurácia metric. H3 sustentada com força especial no regime não-linear — LLM "pega o jeito" mesmo sem augmentação explícita no prompt. Mas o comportamento em $n_{\text{shot}} = 20$ e 40 é não-monotônico, sugerindo plateau com seleção específica de exemplos.

Comparo o LLM com k-NN, Logistic Regression e SVM, todos treinados nos MESMOS exemplos few-shot. Protocolo fundamental: mesmos exemplos, mesmo conjunto de teste. Padrão: LLM compete em n_{shot} baixo, baselines alcançam em n_{shot} alto.

Pedido específico do orientador. Para cada n_{shot} , treino um Perceptron Estruturado nos mesmos exemplos do expert. Achado striking: em peso $\times$ altura $n_{\text{shot}} = 5$, Perceptron 91,7% supera LLM 77,8% por 14 pontos percentuais. Quando há dados rotulados, métrica clássica é competitiva.

Fechando o Bloco 2. H3 sustentada com forte ganho 0 para 5-shot. H4 refutada em n_{shot} pequeno mas revertida em $n_{\text{shot}} = 40$ — diagnóstico mais sofisticado. Baselines clássicos competitivos. Recency bias e viés de classes marginais.

Bloco 3: estudo de caso real, peso $\times$ altura, base do orientador. É onde aplicamos tudo o que vimos em dados reais com 100 amostras e fronteira elíptica conhecida. Pergunta: priors semânticos do LLM ajudam ou atrapalham?

Base enviada pelo orientador no e-mail das 19:15 do dia 30 de abril. 100 amostras, 50 homens e 50 mulheres. Features normalizadas em $[-1, 1]$. Classificador ótimo bayesiano fornecido pelo professor tem forma elíptica.

Duas variantes. NEUTRA: features = (x_1, x_2) , LLM não sabe que é peso e altura. SEMÂNTICA: features = (peso, altura), LLM ativa priors. Pergunta em ambas: homem ou mulher.

Aprendo W em três configurações. O gráfico tem quatro painéis: fidelidade e acurácia, pelos dois algoritmos.

Tabela com prompt semântico. Fidelidade da métrica NNLS sobe de 56% em R2 para 72% em R4, enquanto a acurácia REAL fica em 75-84%. Em prompt NEUTRO, acurácia REAL despenca para 30-40% — LLM sem prior semântico está perdido.

Em prompt NEUTRO, fidelidade sobe de 60% em R2 para 75% em R4, mas acurácia REAL não mexe. A métrica com 4 pesos para 70 amostras de treino decora o ruído das decisões erradas do LLM. Overfitting clássico. Subir `n_feat` só vale com não-linearidade real E dados suficientes.

Tabela: LLM com prompt semântico atinge 99% de acurácia _metric em 40-shot — aprende quase perfeitamente. Em 5-shot, Perceptron baseline com 91,7% supera LLM 77,8% — com dados rotulados poucos, métrica clássica é a melhor opção. Em $n_{\text{shot}} \geq 10$ o LLM passa o Perceptron.

Curvas separadas por variante. Semântico domina neutro em todas as comparações. Em 40-shot semântico atinge 99% e neutro fica em 87%.

Pedido específico do orientador. Mostro o gráfico com bar chart comparando LLM e Perceptron baseline em cada n_{shot} .

Tabela final. Em peso/altura semântico LLM ganha. Em x_1/x_2 neutro ambos são igualmente ruins. Em meia-lua o Perceptron baseline com augmentação bate o LLM in-context. Mensagem honesta para a banca: vantagem do LLM vem dos priors do treinamento massivo, não do mecanismo de ICL per se.

Atende adendo do e-mail das 22:06. Mostro lado a lado peso $\times$ altura com prompt semântico e neutro, ambos 5-shot da seed 42. Os mesmos pontos são classificados de forma muito diferente. Evidência visual do efeito do prior semântico.

Priors semânticos são reais: peso/altura 89% `acc_real`, x_1/x_2 apenas 36%. R4 recupera a forma teórica mas overfit-a com 100 amostras. Em 5-shot peso/altura, Perceptron baseline supera LLM por 14 p.p. — com dados rotulados, métrica clássica é competitiva.

Tabela final cruzando os 3 blocos. A/B/C lineares: ganho zero, esperado. Meia-lua: ganho positivo de 7,6 p.p. R4 fidelidade. Peso $\times$ altura: ganho de 16 p.p. na fidelidade mas -25 p.p. na acurácia REAL em prompt neutro — overfit pesado com 100 amostras.

Tabela: fidelidade Problema A $75,1\% \pm 8,4$, Consist B $65,4\% \pm 7$, Consist C $79,5\% \pm 7,1$. Desvios menores que 8 p.p. entre seeds. H5 sustentada.

Few-shot vs zero-shot na Problema E euclid: delta 0,38. Easy vs Hard em 5-shot: delta 0,26 (refutação H4). Em 40-shot inverte: $-0,14$. R2 vs R4 na meia-lua: $+0,08$. Cosseno entre W's: acima de 0,95. Diagnóstico de gamma no Perceptron: figura final_10 mostra convergência monotônica de gamma_lo.

Três testes adicionais. Point-biserial entre confiança do LLM e acerto: $r = 0,42$, $p < 0,001$. Mann-Whitney comparando margem em acertos vs erros: $p < 0,001$. Kruskal-Wallis sobre ordem de exemplos: $p \approx 0,55$ n.s.

Síntese por bloco. Bloco 1: H1 sustentada parcial, H2 sustentada. Bloco 2: H3 sustentada, H4 refutada com reversão. Transversal: H5 sustentada. Quatro das cinco sustentadas com nuances importantes.

Seis conclusões. Um: critério decisório estável no LLM, ortogonal a correção. Dois: few-shot amplifica. Três: LLM aprende critério via ICL mas baselines são competitivos. Quatro: easy supera hard em pouco mas hard reverte em 40-shot — achado novo nesta execução. Cinco: R3/R4 só ajuda em não-linear E com dados. Seis: vantagem do LLM em peso $\times$ altura vem dos priors semânticos.

Métrica diagonal não captura interações. 2D apenas. Peso $\times$ altura tem só 100 amostras. 1 LLM testado. Sem controle de versão do LLM entre execuções. Honestidade nas limitações é o que separa pesquisa de marketing.

Encerro agradecendo o orientador Prof. Raul Fonseca Neto, o PPGCC da UFJF e minha família. Estou à disposição para perguntas. Tenho dez slides de apêndice técnico cobrindo formulação matemática, overfitting, literatura e reprodutibilidade — inclusive o diagnóstico da busca binária de gamma que o professor pediu.

Para perguntas matemáticas. Formulação primal Eq. 28: $\max \gamma$ sujeito à restrição de margem com relaxação $\lambda \alpha_i$. Regra de correção Eq. 29. Atualização da margem Eq. 31. Hiperparâmetros do artigo: $\eta = 0,001$, C entre 0,1 e 1,0.

NNLS: $\min \|Aw - b\|^2$ com $w \geq 0$. Linha i de A tem diferenças de quadrados aos centróides. b vetor de uns. Método de conjunto ativo de Lawson-Hanson 1974. Sem hiperparâmetros, determinístico, ótimo global.

Perceptron e NNLS são empiricamente equivalentes nesta execução: cosseno acima de 0,95 entre os W 's estimados. Robustez do achado ao método de estimação.

Caso emblemático: peso $\times$ altura com prompt neutro x_1/x_2 , 4 features. Fidelidade 75% mas acurácia REAL apenas 40%. Métrica decora ruído das decisões erradas do LLM. Mitigações: L1/L2, mais dados, CV.

Conexões com literatura. Ahuja & Orlin 2001. Schultz & Joachims 2003. Coelho, Borges, Fonseca Neto CILAMCE 2017. Diferenciação: aplicação a LLMs, black-box, independente de arquitetura.

Brown et al. 2020 GPT-3 e few-shot. Min et al. 2022 ground truth doesn't matter. Wei et al. 2022 emergent abilities. Lyu et al. 2023 prompts canônicos — alinhado com refutação de H4 em few-shot pequeno.

Xing et al. 2002 original. Davis et al. 2007 ITML. Schultz & Joachims 2003 SVM-style. Justificativa diagonal Liu et al. 2010, Kedem et al. 2012.

Comando único para reprodução. Mostro o gráfico do diagnóstico da busca binária de gamma que o professor pediu na reunião em torno de 9 minutos: `gamma_lo` monotônico crescente confirma convergência em todas as execuções.

Honestamente: o LLM evolui silenciosamente entre execuções. Temperatura zero NÃO garante determinismo — batching em GPU produz diferenças residuais. Usamos 3 seeds vezes 3 reps para 9 observações por configuração. Estocasticidade residual típica menor que 2% entre reps da mesma seed.